Information Retrieval

Practical Manual

Prepared By

Ms. Poornima Nadar

Assistant Professor

Department of Computer Science

Course: Information Retrieval

Programme: B.Sc. Computer Science

Semester: VI

Academic Year: 2025 – 2026

INDEX

Sr.No	Topics	Date	Signature
1	**Document Indexing and Retrieval** • Implement an inverted index construction algorithm. • Build a simple document retrieval system using the constructed index.		
2	**Retrieval Models** • Implement the Boolean retrieval model and process queries. • Implement the vector space model with TF-IDF weighting and cosine similarity.		
3	**Spelling Correction in IR Systems** • Develop a spelling correction module using edit distance algorithms. • Integrate the spelling correction module into an information retrieval system		
4	**Evaluation Metrics for IR Systems** • Calculate precision, recall, and F-measure for a given set of retrieval results. • Use an evaluation toolkit to measure average precision and other evaluation metrics.		
5	**Text Categorization** • Implement a text classification algorithm (e.g., Naive Bayes or Support Vector Machines). • Train the classifier on a labelled dataset and evaluate its performance.		
6	**Clustering for Information Retrieval** • Implement a clustering algorithm (e.g., K-means or hierarchical clustering). • Apply the clustering algorithm to a set of documents and evaluate the clustering results.		
7	**Web Crawling and Indexing** • Develop a web crawler to fetch and index web pages. • Handle challenges such as robots.txt, dynamic content, and crawling delays.		
8	**Link Analysis and PageRank** • Implement the PageRank algorithm to rank web pages based on link analysis. • Apply the PageRank algorithm to a small web graph and analyze the results.		

PRACTIAL 1

Aim: Document Indexing and Retrieval

A. Implement an inverted index construction algorithm. Build a simple document retrieval system using the constructed index.

Code:

```
import re
from collections import defaultdict

class DocumentRetrievalSystem:
    def __init__(self):
        self.index = defaultdict(set)
        self.documents = {}

    def tokenize(self, text):
        return re.findall(r'\b\w+\b', text.lower())

    def index_document(self, doc_id, text):
        tokens = self.tokenize(text)
        for token in tokens:
            self.index[token].add(doc_id)
        self.documents[doc_id] = text

    def retrieve_documents(self, query):
        tokens = self.tokenize(query)
        relevant_doc_ids = set()

        for token in tokens:
            if token in self.index:
                if not relevant_doc_ids:
                    relevant_doc_ids.update(self.index[token])
                else:
                    relevant_doc_ids.intersection_update(self.index[token])

        return [(doc_id, self.documents[doc_id]) for doc_id in
relevant_doc_ids]

# Example usage
doc_retrieval = DocumentRetrievalSystem()
doc_retrieval.index_document(1, "This is the first Document")
doc_retrieval.index_document(2, "This is the second Document")
doc_retrieval.index_document(3, "Another document is here")

query = input("Enter your query: ")
results = doc_retrieval.retrieve_documents(query)

print("Query:", query)
for doc_id, text in results:
    print(f"Document {doc_id}: {text}")
```

Output:

```
Enter your query: first
Query: first
Document 1: This is the first Document
```

PRACTIAL 2

Aim: Retrieval Models

A. Implement the Boolean retrieval model and process queries. Implement the vector space model with TF-IDF weighting and cosine similarity.

Code:

```
import pandas as pd
from sklearn.feature_extraction.text import CountVectorizer

print("Demonstration of Boolean Retrieval Model using Bitwise operations on
Term Document Incidence Matrix of a corpus")

# Corpus / Documents
corpus = {
    'this is the first document.',
    'this document is the second document.',
    'And this is the third one.',
    'Is this the first document?'
}

print("\nThe corpus is:")
print(corpus)

# Creating term-document matrix
vectorizer = CountVectorizer()
x = vectorizer.fit_transform(corpus)

# Convert matrix to DataFrame
df = pd.DataFrame(x.toarray(), columns=vectorizer.get_feature_names_out())
print("\nThis generated DataFrame is:")
print(df)

# Query processing
print("\nQuery processing on the term-document incidence matrix:")

# 1. this AND first
print("\n1. Find all document ids for query: 'this AND first'")
alldata = df[(df['this'] == 1) & (df['first'] == 1)]
print("Document ids where both 'this' and 'first' terms are present:",
alldata.index.tolist())

# 2. this OR first
print("\n2. Find all document ids for query: 'this OR first'")
ordata = df[(df['this'] == 1) | (df['first'] == 1)]
print("Document ids where either 'this' or 'first' terms are present:",
ordata.index.tolist())

# 3. NOT and
```

```
print("\n3. Find all document ids for query: 'NOT and'")
notdata = df[(df['and'] != 1)]
print("Document ids where 'and' term is not present:", notdata.index.tolist())
```

Output:

```
Demonstration of Boolean Retrieval Model using Bitwise operations on Term Docume
nt Incidence Matrix of a corpus

The corpus is:
{'Is this the first document?', 'And this is the third one.', 'this document is
the second document.', 'this is the first document.'}

This generated DataFrame is:
   and  document  first  is  one  second  the  third  this
0    0         1      1   1    0       0    1      0     1
1    1         0      0   1    1       0    1      1     1
2    0         2      0   1    0       1    1      0     1
3    0         1      1   1    0       0    1      0     1

Query processing on the term-document incidence matrix:

1. Find all document ids for query: 'this AND first'
Document ids where both 'this' and 'first' terms are present: [0, 3]

2. Find all document ids for query: 'this OR first'
Document ids where either 'this' or 'first' terms are present: [0, 1, 2, 3]

3. Find all document ids for query: 'NOT and'
Document ids where 'and' term is not present: [0, 2, 3]
```

B. Implement the vector space model with TF-IDF weighting and cosine similarity.

Code:

```
# Practical 2b: Vector Space Model using TF-IDF and Cosine Similarity

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

documents = [
    "this is the first document",
    "this document is the second document",
    "and this is the third one",
    "is this the first document"
]

# Vectorize using TF-IDF
tfidf_vectorizer = TfidfVectorizer()
tfidf_matrix = tfidf_vectorizer.fit_transform(documents)

print("TF-IDF Matrix:")
print(tfidf_matrix.toarray())

# Example query similarity
query = ["this is the document"]
query_vector = tfidf_vectorizer.transform(query)

# Calculate cosine similarity
cos_sim = cosine_similarity(query_vector, tfidf_matrix)

print("\nCosine similarity scores for each document:")
print(cos_sim)
```

Output:

```
TF-IDF Matrix:
[[0.         0.46979139 0.58028582 0.38408524 0.         0.
  0.38408524 0.         0.38408524]
 [0.         0.6876236  0.         0.28108867 0.         0.53864762
  0.28108867 0.         0.28108867]
 [0.51184851 0.         0.         0.26710379 0.51184851 0.
  0.26710379 0.51184851 0.26710379]
 [0.         0.46979139 0.58028582 0.38408524 0.         0.
  0.38408524 0.         0.38408524]]

Cosine similarity scores for each document:
[[0.81441289 0.79434606 0.37790643 0.81441289]]
```

PRACTIAL 3

Aim: Spelling Correction in IR Systems

A. Develop a spelling correction module using edit distance algorithms. Integrate the spelling correction module into an information retrieval system

Code:

```
# Practical 3: Spelling Correction and Integration with IR System

# ------------------ Part A: Levenshtein Distance ------------------

def levenshtein_distance(str1, str2):
    len_str1 = len(str1) + 1
    len_str2 = len(str2) + 1

    # Initialize matrix
    matrix = [[0 for _ in range(len_str2)] for _ in range(len_str1)]

    # Initialize first row and first column
    for i in range(len_str1):
        matrix[i][0] = i
    for j in range(len_str2):
        matrix[0][j] = j

    # Fill the matrix
    for i in range(1, len_str1):
        for j in range(1, len_str2):
            cost = 0 if str1[i-1] == str2[j-1] else 1
            matrix[i][j] = min(
                matrix[i-1][j] + 1,      # Deletion
                matrix[i][j-1] + 1,      # Insertion
                matrix[i-1][j-1] + cost  # Substitution
            )

    return matrix[len_str1 - 1][len_str2 - 1]

def suggest_correction(word, word_list):
    # Compute distance for each word in dictionary
    distances = [(w, levenshtein_distance(word, w)) for w in word_list]

    # Sort by distance
    distances.sort(key=lambda x: x[1])

    # Return the word with minimum distance
    return distances[0][0]

# ------------------ Part B: Integration with IR System ------------------

def retrieve_information(query, dictionary):
```

```
    query_words = query.split()
    corrected_words = [suggest_correction(word, dictionary) for word in
query_words]

    corrected_query = ' '.join(corrected_words)

    # Simulated retrieval stage
    print(f"Retrieving information for corrected query: '{corrected_query}'")

# ------------------ Main Program ------------------

if __name__ == "__main__":

    # Test for spelling correction (3a demonstration)
    input_word = "poonimo"
    word_list = ["hello", "world", "python", "spell", "correct", "algorithm",
"poornima", "poohnima"]

    suggested_correction = suggest_correction(input_word, word_list)
    print(f"Suggested correction for '{input_word}': {suggested_correction}")

    print("\n----------------------------\n")

    # Demo for IR Integration (3b demonstration)
    user_query = "speling correctin algorithm"
    dictionary = ["spelling", "correction", "algorithm", "information",
"retrieval", "system"]

    retrieve_information(user_query, dictionary)
```

Output:

```
Suggested correction for 'poonimo': poornima

----------------------------

Retrieving information for corrected query: 'spelling correction algorithm'
```

PRACTIAL 4

Aim: Evaluation Metrics for IR Systems

A. Calculate precision, recall, and F-measure for a given set of retrieval results.

Code:

```
# Practical 4a: Evaluation Metrics (Precision, Recall, F-measure)

def calculate_metrics(retrieved_set, relevant_set):
    true_positive = len(retrieved_set.intersection(relevant_set))
    false_positive = len(retrieved_set.difference(relevant_set))
    false_negative = len(relevant_set.difference(retrieved_set))

    print("True Positive:", true_positive)
    print("False Positive:", false_positive)
    print("False Negative:", false_negative)

    precision = true_positive / (true_positive + false_positive)
    recall = true_positive / (true_positive + false_negative)
    f_measure = (2 * precision * recall) / (precision + recall)

    return precision, recall, f_measure


# Example sets
retrieved_set = set(["doc1", "doc2", "doc3"])
relevant_set = set(["doc1", "doc4"])

precision, recall, f_measure = calculate_metrics(retrieved_set, relevant_set)

print(f"Precision: {precision}")
print(f"Recall: {recall}")
print(f"F-measure: {f_measure}")
```

Output:

```
True Positive: 1
False Positive: 2
False Negative: 1
Precision: 0.3333333333333333
Recall: 0.5
F-measure: 0.4
```

B. Use an evaluation toolkit to measure average precision and other evaluation metrics.

Code:

```
# Practical 4b: Evaluation using Toolkit (Average Precision Score)

from sklearn.metrics import average_precision_score

# Ground truth relevance values
y_true = [0, 1, 1, 0, 1, 1]

# System-generated scores / ranking scores
y_scores = [0.1, 0.4, 0.35, 0.8, 0.65, 0.9]

average_precision = average_precision_score(y_true, y_scores)

print(f"Average precision-recall score: {average_precision}")
```

Output:

```
Average precision-recall score: 0.8041666666666667
```

PRACTIAL 5

Aim: Text Categorization

A. Implement a text classification algorithm (e.g., Naive Bayes or Support Vector Machines). Train the classifier on a labelled dataset and evaluate its performance.

Code:

```
# Practical-5: Text Categorization using Naive Bayes
# Program: 5(a)

import numpy as np
import re

class NaiveBayesClassifier:
    def __init__(self):
        self.vocab = []
        self.positive_counts = None
        self.negative_counts = None
        self.positive_prior = 0.0
        self.negative_prior = 0.0

    # Step 1: Preprocess text (tokenization + lowercase)
    def preprocess_review(self, review):
        tokens = re.findall(r"\b\w+\b", review.lower())
        return tokens

    # Step 2: Convert text into a vector (Bag of Words)
    def vectorize_review(self, tokens):
        vector = np.zeros(len(self.vocab))
        for word in tokens:
            if word in self.vocab:
                vector[self.vocab.index(word)] += 1
        return vector

    # Step 3: Train the classifier
    def train(self, X_train, y_train):
        # Build vocabulary
        for review in X_train:
            tokens = self.preprocess_review(review)
            self.vocab.extend(tokens)
        self.vocab = list(set(self.vocab))

        # Initialize word counts
        self.positive_counts = np.zeros(len(self.vocab))
        self.negative_counts = np.zeros(len(self.vocab))

        # Vectorize training data
        X_train_vectorized = [
            self.vectorize_review(self.preprocess_review(review))
```

```
            for review in X_train
        ]

        positive_count = 0

        # Count word frequencies per class
        for i in range(len(X_train_vectorized)):
            if y_train[i] == 'positive':
                self.positive_counts += X_train_vectorized[i]
                positive_count += 1
            else:
                self.negative_counts += X_train_vectorized[i]

        # Calculate priors
        self.positive_prior = positive_count / len(y_train)
        self.negative_prior = 1 - self.positive_prior

    # Step 4: Predict class for test data
    def predict(self, X_test):
        predictions = []
        for review in X_test:
            review_vectorized = self.vectorize_review(
                self.preprocess_review(review)
            )

            # Likelihood calculation
            positive_likelihood = np.prod(
                (self.positive_counts / sum(self.positive_counts)) **
review_vectorized
            ) * self.positive_prior

            negative_likelihood = np.prod(
                (self.negative_counts / sum(self.negative_counts)) **
review_vectorized
            ) * self.negative_prior

            if positive_likelihood > negative_likelihood:
                predictions.append('positive')
            else:
                predictions.append('negative')
        return predictions


# ------------------- Training Data -------------------
positive_reviews = [
    "The movie was amazing, I like great acting and an engaging plot."
]

negative_reviews = [
    "Terrible! Would not recommend it to anyone."
]

X_train = positive_reviews + negative_reviews
```

```python
y_train = ['positive'] * len(positive_reviews) + ['negative'] *
len(negative_reviews)

# ------------------- Testing Data -------------------
X_test = [
    "The acting was superb!"
]

y_test = ['positive']

# ------------------- Model Execution -------------------
classifier = NaiveBayesClassifier()
classifier.train(X_train, y_train)

predictions = classifier.predict(X_test)
print("Predicted class for the new review:", predictions[0])

accuracy = sum(
    1 for true_label, predicted_label in zip(y_test, predictions)
    if true_label == predicted_label
) / len(y_test)

print("Accuracy:", accuracy)
```

Output:

```
Predicted class for the new review: positive
Accuracy: 1.0
```

PRACTIAL 6

Aim: Clustering for Information Retrieval

A. Implement a clustering algorithm (e.g., K-means or hierarchical clustering). Apply the clustering algorithm to a set of documents and evaluate the clustering results.

Code:

```
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

# Sample documents
documents = [
    "Machine learning is the study of computer algorithms that improve
automatically through experience.",
    "Deep learning is a subset of machine learning.",
    "Natural language processing is a field of artificial intelligence.",
    "Computer vision is a field of study that enables computers to interpret
and understand the visual world.",
    "Reinforcement learning is a type of machine learning algorithm that
teaches an agent how to make decisions in an environment by rewarding desired
behaviors.",
    "Information retrieval is the process of obtaining information from a
collection of documents.",
    "Text mining is the process of deriving high-quality information from
text.",
    "Data clustering is the task of dividing a set of objects into groups.",
    "Hierarchical clustering builds a tree of clusters.",
    "K-means clustering is a method of vector quantization."
]

# Convert documents into TF-IDF vectors
vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(documents)

# Number of clusters
k = 3

# Apply K-means clustering
kmeans = KMeans(n_clusters=k, random_state=42)
kmeans.fit(X)

# Evaluate clustering using Silhouette Score
silhouette_avg = silhouette_score(X, kmeans.labels_)
print("Silhouette Score:", silhouette_avg)

# Print documents in each cluster
for i in range(k):
```

```
    cluster_docs_indices = np.where(kmeans.labels_ == i)[0]
    cluster_docs = [documents[idx] for idx in cluster_docs_indices]
    print(f"\nCluster {i+1}:")
    for doc in cluster_docs:
        print("-", doc)
```

Output:

```
= RESTART: E:/POORNIMA TR 2025-26/2025-26 NOVEMBER- FEBRAURY/Information_Retrieval/PRACTIALS/PRACTIAL
6.py
Silhouette Score: 0.04985132928440607

Cluster 1:
- Natural language processing is a field of artificial intelligence.
- Computer vision is a field of study that enables computers to interpret and understand the visual wo
rld.

Cluster 2:
- Information retrieval is the process of obtaining information from a collection of documents.
- Text mining is the process of deriving high-quality information from text.
- Data clustering is the task of dividing a set of objects into groups.
- Hierarchical clustering builds a tree of clusters.
- K-means clustering is a method of vector quantization.

Cluster 3:
- Machine learning is the study of computer algorithms that improve automatically through experience.
- Deep learning is a subset of machine learning.
- Reinforcement learning is a type of machine learning algorithm that teaches an agent how to make dec
isions in an environment by rewarding desired behaviors.
```

PRACTIAL 7

Aim: Web Crawling and Indexing

A. Develop a web crawler to fetch and index web pages. • Handle challenges such as robots.txt, dynamic content, and crawling delays.

Code:

```
import requests
from bs4 import BeautifulSoup
import time
from urllib.parse import urlparse

class WebCrawler:
    def __init__(self):
        self.visited_urls = set()

    def crawl(self, url, depth=2, delay=1):
        if depth == 0 or url in self.visited_urls:
            return

        if not self.is_allowed_by_robots(url):
            print(f"Skipping {url} due to robots.txt rules")
            return

        try:
            response = requests.get(url)
            if response.status_code == 200:
                soup = BeautifulSoup(response.text, 'html.parser')
                self.index_page(url, soup)
                self.visited_urls.add(url)

                for link in soup.find_all('a'):
                    new_url = link.get('href')
                    if new_url and new_url.startswith('http'):
                        time.sleep(delay)
                        self.crawl(new_url, depth - 1, delay)

        except Exception as e:
            print(f"Error crawling {url}: {e}")

    def is_allowed_by_robots(self, url):
        parsed = urlparse(url)
        robots_url = f"{parsed.scheme}://{parsed.netloc}/robots.txt"

        try:
            response = requests.get(robots_url)
            if response.status_code == 200 and "Disallow: /" in response.text:
                return False
        except:
            pass
```

```
        return True

    def index_page(self, url, soup):
        title = soup.title.string if soup.title else "No title"
        paragraph = soup.find('p').get_text() if soup.find('p') else "No
paragraph found"

        print(f"\nIndexing: {url}")
        print(f"Title: {title}")
        print(f"First Paragraph: {paragraph}")
        print("-" * 40)

if __name__ == "__main__":
    crawler = WebCrawler()
    crawler.crawl("https://www.example.com")
```

Output:

```
Indexing: https://www.example.org
Title: Example Domain
First Paragraph: This domain is for use in documentation examples without needin
g permission. Avoid use in operations.Learn more
----------------------------------------

Indexing: https://iana.org/domains/example
Title: Example Domains
First Paragraph: As described in RFC 2606 and RFC 6761, a
number of domains such as example.com and example.org are maintained
for documentation purposes. These domains may be used as illustrative
examples in documents without prior coordination with us. They are not
available for registration or transfer.
----------------------------------------
```

PRACTIAL 8

Aim: Link Analysis and PageRank

A. Implement the PageRank algorithm to rank web pages based on link analysis. Apply the PageRank algorithm to a small web graph and analyze the results.

Code:

```
def pagerank(graph, damping_factor=0.85, epsilon=1.0e-8, max_iterations=100):
    """
    Compute the PageRank of nodes in the graph.

    :param graph: Dictionary where keys are pages and values are lists of
outgoing links
    :param damping_factor: Probability of following a link
    :param epsilon: Convergence threshold
    :param max_iterations: Maximum iterations
    :return: Dictionary of PageRank scores
    """

    num_nodes = len(graph)

    # Initialize PageRank scores
    pagerank_scores = {node: 1.0 / num_nodes for node in graph}

    for _ in range(max_iterations):
        new_pagerank_scores = {}
        max_diff = 0

        for node in graph:
            # Base PageRank value
            new_pagerank = (1 - damping_factor) / num_nodes

            # Add contributions from incoming links
            for referring_page, links in graph.items():
                if node in links:
                    num_outlinks = len(links)
                    new_pagerank += (
                        damping_factor * pagerank_scores[referring_page] /
num_outlinks
                    )

            new_pagerank_scores[node] = new_pagerank
            diff = abs(new_pagerank - pagerank_scores[node])
            max_diff = max(max_diff, diff)

        pagerank_scores = new_pagerank_scores

        # Check convergence
        if max_diff < epsilon:
```

```
            break

    return pagerank_scores
web_graph = {
    'A': ['B', 'C'],
    'B': ['C'],
    'C': ['A']
}
pagerank_scores = pagerank(web_graph)

sorted_scores = sorted(
    pagerank_scores.items(),
    key=lambda x: x[1],
    reverse=True
)

print("PageRank Scores:")
for node, score in sorted_scores:
    print(f"{node}: {score:.4f}")
```

Output:

```
= RESTART: E:/POORNIMA TR 2025-
al/PRACTIALS/PRACTIAL 8.py
PageRank Scores:
C: 0.3974
A: 0.3878
B: 0.2148
```